

Multi-Server Doubly Efficient PIR in the Classical Model and Beyond

Arthur Lazzaretti¹, Zeyu Liu¹, Ben Fisch¹, Peihan Miao² and Charalampos Papamanthou¹ *

¹Yale University

²Brown University

Abstract. A Doubly Efficient Private Information Retrieval (DEPIR) is defined as a Private Information Retrieval (PIR) scheme with both near-linear server-side preprocessing time and sublinear query time. For many years, it has been a strong open question to devise a DEPIR scheme from standard assumptions. Very recently, Lin et al. (STOC '23) devised the first single-server DEPIR scheme from standard assumptions. However, their work left one major question open: can we build a DEPIR scheme in the multi-server setting, without relying on heavy cryptographic machinery?

In this work, we discuss two different settings for multi-server DEPIR.

In the non-colluding, non-communicating servers setting, we propose the first doubly efficient multi-server PIR scheme. Our scheme is information-theoretic. For a database of size N , our scheme allows servers to answer an infinite number of client queries in $N^{o(1)}$ time, after a single preprocessing phase which takes $N^{1+o(1)}$ time. Our result is only a $N^{o(1)}$ factor from the lower bound proven by Persiano and Yeo (SODA '22) for this setting. In addition, we devise a second information-theoretic PIR scheme in this setting which pushes the state of the art for the setting where the number of servers is more constrained. It achieves equally fast query times as our first scheme above, and a preprocessing time of $N^{2+o(1)}$. This scheme uses $O(\log N)$ servers. Both schemes don't require any cryptographic primitives.

Furthermore, in the setting where the servers are additionally allowed to communicate and keep a global state (which is similarly required by PIR allowing database updates), we show a family of information-theoretic DEPIR schemes which achieve $N^{\text{polylog} N}$ preprocessing time, and $\log^3 N$ communication and server time, for any number of servers k greater than three. We also discuss how introducing more servers or computational assumptions in this setting may improve concrete efficiency of the PIR. This setting of communicating servers requires online communication between servers to answer queries, in contrast to the standard multi-server PIR setting where servers only communicate to coordinate the database contents and do not communicate at query time.

* Authors are listed alphabetically among students, followed by the advisors.

Contacts: arthur.lazzaretti@yale.edu, zeyu.liu@yale.edu, benjamin.fisch@yale.edu, peihan_miao@brown.edu, charalampos.papamanthou@yale.edu

1 Introduction

Private Information Retrieval (PIR) was first introduced three decades ago by Chor et al. [15] in the multi-server, information-theoretic setting. In PIR, a client holds an index $i \in [N]$, and k , non-colluding servers hold the same database $\text{DB} \in \{0,1\}^N$. The goal of the protocol (informally) is to devise a way for the client to learn $\text{DB}[i]$ without either server learning any information about the client's desired index i . PIR has been used as a building block for many privacy-preserving applications such as private web searches [39], privacy-enhanced Tor [65], private ad serving [1], among others.

Since the problem's original inception, many works have pushed our understanding of the PIR problem in this same setting [15,14,7,78,81] (a non-exhaustive list) as well as many different scenarios; single-server computational PIR [49], two-server computational PIR [32,12,53,54] among others [9,83,51]. Aside from the number of servers and the computational assumption on the adversary, we can also consider preprocessing or non-preprocessing schemes. Within preprocessing schemes, there are two well-known approaches, which we will denote server preprocessing and client preprocessing.

In client preprocessing, the server and client run a joint preprocessing phase after which the client stores a secret hint which can be used to perform queries to the server later in time sublinear in the database size. This requires each client to run an expensive (linear time) preprocessing phase with the server, which means the server work to server m clients is at least $m \cdot N$, where N is the database size.

On the other hand, in server preprocessing, the server runs a one-time preprocessing phase by itself and stores additional bits server-side, which it can then use to answer queries from any client in time sublinear in the database size. In this work, we focus on the server preprocessing setting (we cover a broad overview of the other settings in Section 1.2).

The server preprocessing paradigm has a clear advantage with respect to total server time, in the sense that the overhead for serving many clients is considerably smaller. The first family of server preprocessing schemes was introduced by Beimel et al. [7] in 2000 in the multi-server setting. They present schemes for constant and polylogarithmic number of servers, with different trade-offs, as well as a number of lower bounds. Notably, in its Journal of Cryptology version [8], the authors present the first multi-server scheme (with a polylogarithmic number of servers) in the server preprocessing model which achieves $\text{polylog}(N)$ query time and $\text{polylog}(N)$ communication after a preprocessing phase running in time $\text{poly}(N)$.

Persiano and Yeo [71] recently tighten the lower bounds shown by Beimel et al., showing that any server preprocessing scheme in the single server setting with query time $\Omega(t)$ must have the server store a hint of at least size $\Omega(N \log N/t)$, for any number of servers.

Very recently, in a seminal paper by Lin et al. [57] which makes use of a polynomial preprocessing algorithm by Kedlaya and Umans [47] to achieve a PIR scheme which preprocesses in time $O_\lambda(N^{1+o(1)})$ and stores $O_\lambda(N^{1+o(1)})$

after which it can serve PIR queries in time $N^{o(1)}$.¹ They refer this as the *doubly efficient PIR*: in particular, the server can preprocess the database *once in nearly linear time* (e.g., $N^{1+o(1)}$), and answer an infinite number of queries from the clients very quickly (e.g., $N^{o(1)}$ time).

According to the bound by Persiano and Yeo, this is close to optimal asymptotically (only $N^{o(1)}$ factors away). After the work by Lin et al., two main questions were left open for server preprocessing PIR:

1. Can we achieve similar asymptotics in the multi-server setting without the computational assumption (i.e., information-theoretic) while minimizing the number of servers required?
2. Can we achieve doubly efficient PIR without the use heavy cryptographic primitives such as fully homomorphic encryption (FHE)?

The first question is important theoretically due to the relationship of multi-server information-theoretic PIR with other cryptographic primitives [81,80] and to push our understanding of the problem. The second question was also posed after a follow-up work by Okada et al. [68,69] showed that the scheme by Lin et al., implemented as is, still incurs astronomic empirical costs despite the very nice asymptotic efficiencies.

In this work, we answer both questions in the affirmative by providing novel families of multi-server server preprocessing scheme with near-optimal asymptotics that do not use FHE.

Specifically, we introduce three families of multi-server, server-preprocessing PIR schemes for two different settings. First, we consider the setting of multiple non-colluding and non-communicating servers holding the database. Then, we consider the setting of multi-server server preprocessing PIR schemes where the servers can additionally communicate online (note that while the standard multi-server PIR setting enforces non-communication between servers, offline communication is still required to ensure synchronization between databases).

Since our constructions do not rely on any heavy cryptographic tools (and most constructions require no cryptographic tools at all), our work is notably empirically much more efficient in practice than the work by Lin et al., given that the Algebraic Homomorphic Encryption necessary to build their PIR scheme incurs large overheads [68].

Our work pushes our understanding of multi-server server preprocessing PIR and we believe this may pave the way towards building a truly practical doubly efficient PIR constructions in both the single and multi-server settings.

We conclude our work with thoughts on important open questions to be resolved in future work, as well as briefly discuss the possibility of replacing the non-collusion or honesty assumption with the rational server assumption.

1.1 Our Results

Our results can be categorized in two settings.

¹ Or alternatively preprocess in $N^{1+\epsilon}$ time and query in $\text{polylog}(N)$ time for some constant ϵ .

Scheme	Efficiency			
	# of Servers	Preprocessing Time & Storage	Online Time	Communication
[78]	2	$\text{poly}(N)$	$N/\text{polylog}(N)$	$N^{1/3}$
[8]	k	$\text{poly}(N)$	$O(N^{1/k+\epsilon})$	$O(N^{1/k+\epsilon})$
[30]	k	$\text{poly}(N)$	$O(N^{1-\gamma(k-1)+\epsilon})$	$O(N^{\gamma+\epsilon})$
[8,30]*	$O(\text{polylog}(N))$	$\text{poly}(N)$	$\text{polylog } N$	$\text{polylog } N$
Theorem 2	$\tilde{O}(\log(N))$	$N^{1+o(1)}$	$N^{o(1)}$	$N^{o(1)}$
Theorem 3	$o(\log(N))$	$N^{2+o(1)}$	$N^{o(1)}$	$N^{o(1)}$
Theorem 4**	3	$N\text{polylog } N$	$\log^3(N)$	$\log^3(N)$

Table 1. Comparisons to prior works. N is the number of bits in a database, and k, ϵ are tunable constants in [8]. γ is a tunable constant satisfying $\frac{1}{k+1} \leq \gamma \leq \frac{1}{k}$ in [30]. Our $\tilde{O}(\log(N)) = \frac{\log(N) \log \log(N)}{\log \log \log(N)}$.

* : A more recent version of [30] shows how to achieve a scheme with $N^{1+o(1)}$ preprocessing and storage and $N^{o(1)}$ online time and communication (see Appendix A for more details).

** : This scheme works in the setting where servers are allowed to communicate online and also maintain a global state (i.e., the database updates per query, similar to updatable PIR, which also requires database updates periodically).

New PIR schemes for non-colluding, non-communicating servers. First, we construct two k -server information-theoretic PIR construction with server-side preprocessing. Both are compared with the state-of-the-art prior works in information-theoretic PIR with server-side preprocessing [8,78] and the concurrent work [30] in Table 1.

In particular, our *first* scheme in this setting is a doubly efficient PIR scheme (see Definition 1) with no computational assumptions where the server computation at query time is significantly smaller than N , specifically, $N^{o(1)}$. Also, in order to satisfy the definition of doubly efficient, it means that each server’s preprocessing time is near-linear in the database size N (in particular, we have $N^{1+o(1)}$). These are achieved with roughly $O(\log(N))$ servers.

In our second scheme in this setting, we demonstrate that one can further reduce the number of servers required for the scheme, at the expensive of a more costly preprocessing phase, which is no longer near linear, but $N^{2+o(1)}$. This is the first scheme to achieve $N^{o(1)}$ query time with $\text{poly}(N)$ preprocessing and $o(\log N)$ servers. Note that such efficiency is also not achieved by any prior work with $o(\log(N))$ servers (even *any* $\text{poly}(N)$ preprocessing time).

To transfer the doubly efficient PIR scheme of Lin et al. [57] into the information-theoretic setting, the first challenge is to find a way to hide the query without employing homomorphic encryption. In the multi-server setting, secret sharing has been a solution to hide the client query without the use of encryption [73,5]. However, it is not immediately clear how to secret share the input in a way that preserves the homomorphism necessary to evaluate the polynomial from the server side without blowing up the cost.

We therefore take a slightly different approach, inspired by the elegant result of Woodruff and Yekhanin [78], which removes the necessity for finding a secret sharing scheme with homomorphic properties. In particular, they ask each server to evaluate a single point on the polynomial representing the database, and the client can recover the database point using these evaluations. Following this work, we can apply the polynomial-preprocessing data structure considered in [57]. In more detail, we can let each server preprocess the polynomial in advance, and then evaluate the preprocessed result online.

One immediate issue, however, is that using the result in [57], the resulting scheme has relatively bad efficiency. In particular, we need the preprocessing time to be $\text{poly}(N)$ or we need $\text{polylog}(N)$ servers with some large hidden factor in the exponent (i.e., $\log^r(N)$ for some large r , i.e. $r > 20$).

Thus, instead, We also employ a different polynomial preprocessing technique [10] than the one used in [57]. Such new result introduces some new constraints (e.g., the characteristic of $\mathbb{F}_q$ the polynomial works over p needs to be $d^{o(1)}$ where d is the individual degree of the polynomial), and thus we need to carefully balance how the parameters are chosen to make sure the preprocessing works and achieve the desired efficiency. We expand on these in more detail in Sections 4 and 5.

New PIR schemes in the multi-server setting with communication. In this part of our work, we consider a setting where the database is distributed across multiple servers, and these servers are allowed to communicate and maintain state. Note here that the non-collusion assumption remains. We believe the communication among servers is a reasonable setting. Specifically, since PIR schemes assume the databases on all servers to be identical, communication is required at least to synchronize the data between all servers. The difference is that in this setting, communication is necessary at query time. Similarly, the cost of (global) statefulness is also minor, since most use cases of PIR [4,39] consider dynamic databases and require periodic updates. Thus, being stateful is not an extra requirement for updatable PIR.

Furthermore, in our construction, the cost of updating the state is not larger than the cost of serving the client query, which greatly surpasses the existing updatable PIR constructions [55,27,43].

For this setting, we show that against an unbounded adversary, $k = 3$ servers are sufficient to achieve a DEPIR scheme with $N \text{poly log } N$ preprocessing time and $\log^3(N)$ online communication and server computation time.

This family of PIR construction leverages a cryptographic primitive called distributed oblivious RAM (DORAM) [62]. Interestingly, while there have been a lot of constructions for DORAM [62,82,21,37,22,45,23,25,76,38], to our knowledge, there is no construction that uses DORAM to build PIR. We show that DORAM can be directly used to build PIR with essentially no overhead, and thus any concretely efficient DORAM implies a concretely efficient PIR scheme in this communicating-server setting. Also, increasing the number of servers or

introducing computational assumptions can improve concrete costs. This is discussed in Section 6.

Comparisons with prior information-theoretic PIR works. As shown in Table 1, compared to [8] with constant k number of servers, our construction achieves better efficiency (both for preprocessing and online queries). A similar comparison can be drawn when compared to [78]. Compared to [8] with polylogarithmic number of servers, our construction achieves better preprocessing efficiency, while requiring a smaller number of servers. On the other hand, our online efficiency is slightly worse. Nonetheless, altogether, our work provides a preprocessing time and query time trade-off not achievable before.

Comparison with concurrent and independent work. Some recent concurrent and independent work addresses some of the same problems we tackle in this paper. At a high level, their scheme achieves similar complexities as ours in the settings we share, although some of their constructions generalize to a broader parameter set. Their work does not cover the communicating server setting in Theorem 6, that is unique to our work. We provide a more precise comparison between the two schemes in Appendix A.

1.2 Other Related Works

Client preprocessing PIR was first introduced in the two-server setting by Corrigan-Gibbs and Kogan [20,48,19,86,77,27,52,50,85,55,72]. As aforementioned, in this setting, the client and server jointly run a preprocessing phase and the client stores a hint which it later uses to perform queries in time sublinear in the database size. (A version where the queries run in linear time was introduced by [70] earlier.) Although the server has to run one preprocessing phase per client, one big advantage of this model is that the server stores no additional bits server-side other than the original database. In contrast, server preprocessing schemes can require quite large server storage overheads, up to 100x the size of the original database [7,57]. The original work also showed a lower bound on the efficiency of schemes in client preprocessing model. In specific, if the preprocessing phase outputs a hint of size $O(Q)$, then the query phase must at least in time $O(N/Q)$. Several works have since improved the original work by Corrigan-Gibbs et al. in terms of bandwidth [74], as well as devised single-server client preprocessing schemes with similar techniques and asymptotics [19,51,84,26].

One line of work in server preprocessing not mentioned above, by [13,42] achieved PIR schemes with $\tilde{O}(\text{poly}(N))$ preprocessing time and $\text{polylog}(N)$ both client and server time per query from non-standard assumptions. Holmgren et al. [42] were actually the first to coin the term *doubly efficient PIR*. Both their work and the concurrent work by Boyle et al. [13] also consider the problem of doubly efficient PIR. We do not make comparison with these works because they both rely on non-standard assumptions to build their schemes. Specifically, the scheme in [42] relies on a Hidden Permutation with Noise (HPN) assumption

and the scheme in [13] relies on a permuted puzzles assumption (a toy version of which was broken in subsequent works). We do not currently still have a good grasp on the true level of security these conjectured assumptions provide. Furthermore, these schemes rely on a trusted party to run the preprocessing. Since then, there have been no advances in this line of work which considers a trusted third party to run the preprocessing (even under standard assumptions) and it could also be an avenue towards more practical PIR (although the trusted party assumption is difficult to enforce in practice).

Other works have studied preprocessing that does not reduce asymptotic server load but performs extremely fast in practice in the single server setting [40,64,66,2]

PIR has been used as a building block for many different privacy-preserving applications, such as meta-data hiding messaging [3], private streaming [36], private contact-tracing [41], and more [4,64]. One interesting application of PIR is Oblivious Message Retrieval (OMR), a primitive introduced in [59] to address the private retrieval issue in the setting of privacy-preserving blockchains. In a very recent work [46], Jia et. al developed a multi-server OMR construction that uses PIR as a building block. Therefore, multi-server PIR can also be interesting to blockchain settings.

2 Preliminary

In this section we will cover important primitives and definitions necessary for our work.

Notation. $\mathbb{F}_q$ defines a finite field of size q . The characteristic of a finite field is the smallest positive number of copies of the field’s multiplicative identity (i.e., 1) that will sum to the additive identity (i.e., 0). If q is a prime power (i.e., $q = p^\alpha$ for some prime p and $\alpha \geq 0$), the characteristic of $\mathbb{F}_q$ is p . We use $\tilde{O}(\log(N))$ to represent $O(\log(N)\text{poly log log}(N))$.

2.1 Private Information Retrieval

First, we define doubly efficient private information retrieval (DEPIR) formally. This definition captures the informal requirements of our client always being able to reconstruct the desired database entry at the end of the protocol, as well as the requirement that servers, individually, not learn anything about the query index picked by the client. (This is the standard definition for non-colluding, non-communicating servers.)

Definition 1 (Multi-server DEPIR with server preprocessing [16,57]).

A k -server, server preprocessing Private Information Retrieval scheme for a database of length N consists of four PPT algorithms:

- *a preprocessing function $\text{Preprocess}: \text{DB} \in \{0,1\}^N \rightarrow \widetilde{\text{DB}}$;*

- a query function $\text{Query} : (i \in [N], j \in [k]) \rightarrow q$;
- an answer functions $\text{Answer} : (\widetilde{\text{DB}}, q) \rightarrow A$;
- a reconstruction function $\text{Reconstruct} : a_1, \dots, a_k \rightarrow \{0,1\}$.

These functions should satisfy the correctness and privacy defined below.

Correctness: For every $\text{DB} \in \{0,1\}^N$, let $\widetilde{\text{DB}} = \text{Preprocess}(\text{DB})$. For any $i \in [N]$, let $q_j \leftarrow \text{Query}(i, j)$, and $a_j \leftarrow \text{Answer}(\widetilde{\text{DB}}, q_j)$, where $j \in [k]$, it holds that

$$\Pr[\text{Reconstruct}(i, a_1, \dots, a_k) = \text{DB}[i]] = 1$$

Privacy: For every $i_1, i_2 \in [N]$, $j \in [k]$, and $q \in \mathbf{Q}$ where $\mathbf{Q}$ is the output space of Query ,

$$\Pr[\text{Query}(i_1, j) = q] = \Pr[\text{Query}(i_2, j) = q]$$

Then, a PIR scheme is doubly-efficient if it satisfies the following property.

Doubly efficient: A PIR construction is doubly efficient if it is both Preprocessing efficient and Query efficient, which are defined as follows:

- (Preprocessing efficient) For any database $\text{DB} \in \{0,1\}^N$, the runtime of $\widetilde{\text{DB}} \leftarrow \text{Preprocess}(\text{DB})$ and $|\widetilde{\text{DB}}|$ are both $N^{1+o(1)}$.
- (Query efficient) For any $i \in [N]$, let $q_j \leftarrow \text{Query}(i, j)$, $a_j \leftarrow \text{Answer}(\widetilde{\text{DB}}, q_j)$, for all $j \in [k]$, and $\text{Reconstruct}(i, a_1, \dots, a_k)$, it holds that:
 1. The total runtime of the k calls of Query , Answer and the one call of Reconstruct in total are $N^{o(1)}$.
 2. $|(q_1, \dots, q_k, a_1, \dots, a_k)| = N^{o(1)}$.

Note that this definition naturally captures the multi-query setting since it assumes correctness for any query after the preprocessing. This definition is slightly modified from the original definition presented in [16] (and adapted according to [57] to accommodate the doubly efficient definition). For the double efficiency requirement, we additionally require the preprocessing to be at most $N^{1+o(1)}$ instead of $\text{poly}(N)$ as in prior works.

PIR with communicating and stateful servers. Now, we alternatively define PIR against a set of k communicating servers. The definition above is not sufficient, since the interfaces and privacy do not take communication and statefulness of the servers into consideration. While we maintain the standard definition above for our schemes in Sections 4 and 5, we provide a new definition for the multi-server setting that allows communication.

Our new definition follows from other multi-party computation privacy notions. We allow server interaction, and define the new privacy guarantee (servers learn nothing as long as at least one participating servers is honest). In more detail, the new game asks the adversary to distinguish the view of any subset of the servers (up to $k - 1$ servers) for two (adversarially picked) sequences of queries

This is captured formally below.

Definition 2 (Communicating and Stateful Multi-server DEPIR with server preprocessing).

A k -server, server preprocessing, and server communication allowed, Private Information Retrieval scheme with stateful servers for a database of length N consists of four PPT procedures:

- A preprocessing procedure $\text{Preprocess}(\text{DB} \in \{0,1\}^N) \rightarrow (\widetilde{\text{DB}}_j)_{j \in [k]}$; where server j holds $\widetilde{\text{DB}}_j$;
- a query procedure $\text{Query}(i \in [N]) \rightarrow q_1, \dots, q_k$;
- an interactive answer procedure $\text{Answer}((\widetilde{\text{DB}}_j)_{j \in [k]}, (q_j)_{j \in [k]}) \rightarrow a_1, \dots, a_k$; run between the k servers, servers are stateful and thus this procedure also implicitly outputs updated $\widetilde{\text{DB}}_j$ for $j \in [k]$.
- a reconstruction procedure $\text{Reconstruct}(a_1, \dots, a_k) \rightarrow \{0,1\}$; this takes in the server answers and outputs the queried database bit.

These functions should satisfy the correctness and privacy games below, as well as the efficiency requirement; for any $w, k, N \in \mathbb{N}$, $k \geq 2$.

Correctness. For every $\text{DB} \in \{0,1\}^N$, let $(\widetilde{\text{DB}}_j)_{j \in [k]} = \text{Preprocess}(\text{DB})$. For any $i \in [N]$, let $(q_j)_{j \in [k]} \leftarrow \text{Query}(i)$, and $(a_j)_{j \in [k]} \leftarrow \text{Answer}((\widetilde{\text{DB}}_j)_{j \in [k]}, (q_j)_{j \in [k]})$, it holds that

$$\Pr[\text{Reconstruct}(i, a_1, \dots, a_k) = \text{DB}[i]] \geq 1 - \text{negl}(\lambda)$$

Privacy. The following must hold for every $\text{DB} \in \{0,1\}^N$, and for any two sequences $i_{1,1}, \dots, i_{1,w} \in [N]$ and $i_{2,1}, \dots, i_{2,w} \in [N]$.

For $\alpha \in \{1,2\}$:

- Let $(\widetilde{\text{DB}}_j^\alpha)_{j \in [k]} = \text{Preprocess}(\text{DB})$.
- For $\beta \in [w]$:
 - Let $(q_j^{\alpha,\beta})_{j \in [k]} \leftarrow \text{Query}(i_{\alpha,\beta})$.
 - Let $(a_j^{\alpha,\beta})_{j \in [k]} \leftarrow \text{Answer}((\widetilde{\text{DB}}_j^\alpha)_{j \in [k]}, (q_j^{\alpha,\beta})_{j \in [k]})$.

Then, for any $j \in [k]$, let $\text{View}_j(\alpha)$ denote the view of server j when processing queries for $i_{\alpha,1}, \dots, i_{\alpha,w}$. A PIR scheme is private if it holds that $\text{View}_j(1)$ is indistinguishable from $\text{View}_j(2)$ for all $j \in [k]$.

Doubly Efficient. Finally, a PIR scheme is doubly-efficient if it satisfies the efficiencies denoted in Definition 1 (with interfaces adapted accordingly).

2.2 Polynomial preprocessing

As aforementioned, our (first two) schemes will make use of polynomial preprocessing schemes, which take as input a polynomial and output a data structure that can later be used to evaluate the input polynomial much faster than a standard evaluation. Specifically, a polynomial preprocessing scheme has two algorithms. The first algorithm, preprocessing, takes in a polynomial and outputs a datastructure. The second algorithm, evaluate, takes in this data structure and any point and outputs the evaluation of the polynomial.

We restate the results from Bhargava et al. [10] as follows.²

Theorem 1 (Thm 6.1 in [10]). *For a m -variate polynomial $P : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$ with individual degree $\leq d$, over some finite field $\mathbb{F}_q$ such that $q = p^\alpha$ for some prime $p = d^{o(1)}$ and $\alpha > 0$ then, there exists an algorithm `PolyPreProcess` such that (1) the runtime of `PolyPreProcess`(P) is $(\alpha d p)^m \cdot 4^m \cdot \text{poly}(\alpha, d, m, p)$; (2) let $\tilde{P} \leftarrow \text{PolyPreProcess}(P)$, for any $\vec{x} \in \mathbb{F}_q^m$, $\tilde{P}(\vec{x}) = P(\vec{x})$ and that the runtime of $\tilde{P}(\vec{x})$ is $4^m \cdot \text{poly}(\alpha, d, m, p)$.*

With appropriate parameter choices, we will be able to find a polynomial P that represents our database of N bits such that we can preprocess P into a data structure in time $O(N^{1+o(1)})$ and subsequently evaluate P at any point in time $O(N^{o(1)})$. We dive more into the exact details in Section 4.

2.3 Polynomial Interpolation

Notice that to be able to use the polynomial preprocessing scheme mentioned above in a PIR scheme, we need to find a polynomial that represents the database. For any $x \in \{0,1\}^N$, we need to find a polynomial P such that for any $i \in [N]$, $P(\text{encode}(i)) = x_i$, for some encoding function `encode` to be defined later. This is the problem of polynomial interpolation. Given a set of N points, we need to fit a polynomial to these points. It is well-known that we can find a univariate polynomial of degree N that interpolates N points in time $O(N \log N)$ [57,29]. This result is also extended to the multi-variate polynomial setting, as seen in the theorem below.³

Lemma 1 (Lemma B.1 in [57]). *Let $\mathbb{F}_q$ be a finite field, and let $m \in \mathbb{N}$ be an integer. Let $\{y_{x_1, \dots, x_m} \in \mathbb{F}_q\}_{(x_1, \dots, x_m) \in \mathbb{F}_q^m}$ be any set of d^m values. Then, there is an algorithm that runs in time $O(d^m \cdot m \cdot \text{polylog}(q))$ and recovers the coefficients of a polynomial $f(X_1, \dots, X_m) \in \mathbb{F}_q[X_1, \dots, X_m]$ with individual degree less than d in each variable such that $f(x_1, \dots, x_m) = y_{x_1, \dots, x_m}$ for all $(x_1, \dots, x_m) \in \mathbb{F}_q^m$.*

3 Multi-server PIR by Woodruff and Yekhanin

We first recall in detail the scheme by Woodruff and Yekhanin [78] in the k -server, no preprocessing PIR setting, which elegantly achieves large bandwidth improvements with respect to its predecessors. Our construction greatly relies on this work.

Let $h = 2k - 1$ where recall k is the number of non-colluding servers available in the PIR scheme. We first define an encoding function used in [78] `encode` :

² This is a follow-up work to [47] which is used by [57] in their PIR construction, with slightly different asymptotics and tradeoffs. Note that [11] improves the result from [10] on some fronts, but some technical details make it unusable for our use case.

³ Note that in [57], their lemma is stated in a prime field. However, it easily extends to any finite field, as the uni-variate polynomial interpolation used in its proof extending to multi-variate interpolation can be done in any finite field as shown in [29].

$[N] \rightarrow \mathbb{F}_p^m$ where $m = O(hN^{1/h})$ and $k < p < 2k$. We define $\text{encode}(1), \dots, \text{encode}(N)$ to be distinct points of hamming weight h in $\mathbb{F}_p^m$ with each individual value restricted to $\{0,1\} \subset \mathbb{F}_p$. Such encoding exists if $C(m,h) \geq N$, where C here denotes the number of combinations [6,78]. (This is database-independent but depends on public parameters such as the size of the database and the number of servers.)

Now, suppose the servers also are given along with the database an m -variate polynomial F such that for each $i \in [N]$, $F(\text{encode}(i)) = \text{DB}[i]$ (this can be interpolated in near-linear time).

Note F is a m -variate polynomial with a total degree of $2k - 1$ which maps $\mathbb{F}_p^m \rightarrow \mathbb{F}_p$. Explicitly, F can be written as:

$$F(z_1, \dots, z_m) = \sum_{i=1}^N \left(\text{DB}[i] \cdot \prod_{\text{encode}(i)[\ell]=1} z_\ell \right),$$

where $\text{encode}(i)[\ell]$ denotes the ℓ -th element of the output of $\text{encode}(i)$. One can work out that $F(\text{encode}(i)) = \text{DB}[i]$. Also, since each output of encode has weight h , it follows that also F has total degree at most h (at most h variable are multiplied together per term). From this setup, the client performs a query to an index $i \in [N]$ as follows:

1. Pick $\vec{v}$ uniformly at random from $\mathbb{F}_p^m$.
2. To each server $j \in [k]$, send $q_j = \text{encode}(i + \lambda_j \vec{v})$, where each λ_j is a distinct non-zero element from $\mathbb{F}_q$.
3. Server j computes and returns $F(q_j)$ as well as the partial derivatives $\frac{\partial F}{\partial z_1}(q_j), \dots, \frac{\partial F}{\partial z_m}(q_j)$.

The last part of item 3 refers to the partial derivative of F with respect to variable z_i evaluated at point q_j , for each $i \in [m]$.

First, note that we can define a function $f : \mathbb{F}_q \rightarrow \mathbb{F}_q$ such that $f(\lambda_j) = F(i + \lambda_j \vec{v})$. This function has a degree at most $h = 2k - 1$ because F has total degree $h = 2k - 1$ as explained above. Also note that $f(0) = F(i) = \text{DB}[i]$ by construction.

If it were the case that $h = k - 1$, we could recover the entire function f (and thus $f(0) = \text{DB}[i]$) with each server returning only $F(\text{encode}(i) + \lambda_j \vec{v}) = f(\lambda_j)$ and then interpolating. However, since $h = 2k - 1$, the max degree of our local polynomial f is at most $2k - 1$ and therefore k points are not enough to recover $f(0)$. The observation by [78] is that we can use the fact that to interpolate a polynomial of degree $2k - 1$, it suffices to know k points of the polynomial and the derivative at these same k points. Now, from the partial derivatives returned by each server, we can recover $f'(\lambda_j)$ by applying the chain rule as follows:

$$f'(\lambda_j) = \frac{\partial F(i + \lambda_j \vec{v})}{\partial \lambda} = \sum_{\ell=1}^m \vec{v}_\ell \cdot \frac{\partial F}{\partial z_\ell}(\text{encode}(i) + \lambda_j \vec{v}).$$

Since the client knows the first element in the sum and the server returns the second part, it can therefore compute $f'(\lambda_j)$ for each $j \in [k]$ and use these along with the set of $\{f(\lambda_j)\}_{j \in [k]}$ to compute $f(0) = F(\text{encode}(i) + 0 \cdot \vec{v}) = \text{DB}[i]$.

With this, the client can recover $f(0) = \text{DB}[i]$ the data point with k servers and $O(k^2 \log k N^{1/(2k-1)})$ of communication cost.

This scheme is the starting point for our multi-server scheme with preprocessing. Note that [78] do include mention of preprocessing PIR in their work as well, but the best they can achieve is a query time of $O(N/\log^r N)$ for some constant r . In contrast, we will pair this technique with the polynomial preprocessing techniques to achieve a query time of $N^{o(1)}$.

Below is a lemma shown along the way in [78], which we will need for our correctness proof. Essentially, it helps showing that $f(\lambda_j)$ together of the derivatives $f'(\lambda_j), \dots, f^\ell(\lambda_j)$ are enough to uniquely determine a univariate function f with degree $k \cdot \ell - 1$ for $j \in [k]$ and $k \geq 1$.

Lemma 2 (Lemma 1 in [78]). *Let $f \in \mathbb{F}_q[X]$ where $q = p^\alpha$ for some prime p and $\alpha > 0$, and $\ell \leq p - 1$. Suppose $f(X_0) = f'(X_0) = \dots = f^\ell(X_0) = 0$, then $(X - X_0)^{\ell+1} | f$.*

4 Our First Construction

We now move to construct our multi-server doubly efficient PIR scheme. Recall that doubly efficient refers to almost linear server preprocessing time, and sub-linear client query times. Note that the scheme recalled in Section 3 does not fit the definition since the server performs $\Omega(N)$ work per query.

Then, we pose the question of whether it is possible to utilize the recent idea in [57] to enhance the scheme in [78] preprocessing the polynomial first, and the allow the servers to serve client queries in time sublinear in the database size N . In particular, in [47], the authors show how to construct a data structure that allows one to preprocess an m -variate polynomial with individual degree $< d$ such that the preprocessing takes $\Omega(d^m \cdot m^m)$ time (and storage) and the evaluation of any arbitrary input can be done in $\text{poly}(m, d, \log(q))$ time (used by Lin et al. in the PIR context in [57]).

Unfortunately, naively applying this technique to the scheme by Woodruff and Yekhanin does not achieve the desired results. Specifically, in the scheme presented, the preprocessing takes roughly $O(2^{N^{1/k}})$, since $m \approx N^{1/k}$ and $d = 2$ (each variable only has degree 1) for some constant k . Therefore, the question is: can we achieve (almost) N total preprocessing time and server-side storage while achieving (almost) constant online costs (computation and communication)?

4.1 Substituting the Function

As mentioned, the major issue is that the function evaluated by the server cannot be efficiently preprocessed. Therefore, our first idea is to replace the underlying function.

As shown in [57, Claim 4.2.1], instead, we can first encode each index $i \in [N]$ into a d -ary vector $\vec{i}$ such that $i = \sum_j \vec{i}_j \cdot d^{j-1}$ (via some deterministic encoding algorithm $\text{encode}(\cdot)$). Then, we can interpolate a polynomial P with m variables and individual degree d over some finite field $\mathbb{F}_q$ for $q > d$, where $d^m \geq N$, such that $P(\vec{i}) = \text{DB}[i]$.

In this case, the client instead needs to interpolate a function f with degree $d \cdot m$ (since P has m variables and each variable has degree at most d). To interpolate such a function, the client needs $d \cdot m + 1$ points, which can be obtained by $d \cdot m + 1$ servers. In [57], they provide two sets of parameter settings:

1. In the first setting, we set $d = O(\log^{2/\epsilon} N)$ and set $m = O(\epsilon \log(N) / \log \log(N))$ for some constant ϵ . In this case, the total number of servers needed is $O(\log^2(N))$ if we set $\epsilon = 2$, and the preprocessing takes $N^{1+\epsilon} = N^3$, which is $\text{poly}(N)$. Alternatively, ϵ can be set smaller (e.g. $1/10$), and the preprocessing becomes $N^{1.1}$ which is much closer to linear preprocessing. However, in this case, the number of servers needed is then $\log^{21}(N)$ which is quite large.
2. In the second setting, $d = 2^{O(\sqrt{\log(N)})}$ and $m = O(\sqrt{\log(N)})$. Under this setting, the preprocessing time remains $N^{1+o(1)}$ which is essentially linear. However, the total number of servers required in this case is $2^{\sqrt{\log(N)}}$, which is super-polylogarithmic, and undesirable.

Then, using this framework, it seems impossible to achieve both a polylogarithmic number of servers and an almost linear time preprocessing time. However, we notice that [10] (recalled in Theorem 1) reduces the preprocessing time of the scheme used in [57] from $d^m \cdot m^m$ to $d^m \cdot 4^m$. The requirement to use this scheme is that it only works with finite fields of small characteristic ($p = d^{o(1)}$).

By leveraging this new result, with a careful parameters selection we can achieve all of our desired asymptotics, (1) a polylogarithmic number of servers and (2) almost linear preprocessing time and (3) fast query times.

Specifically, we show in Theorem 2 that with such a new technique, and careful parameter selection, we can reduce the number of servers to $\tilde{O}(\log(N))$, while keeping the preprocessing time $N^{1+o(1)}$, and with a query time of $N^{o(1)}$.

We formally present our scheme in Fig. 1.

Theorem 2. *The Private Information Retrieval scheme in Figure 1 satisfies Definition 1 is doubly-efficient. In particular, it runs with efficiencies:*

- Number of servers: $\tilde{O}(\log(N)) = O(\frac{\log(N) \log \log(N)}{\log \log \log(N)})$.
- Preprocess time and server storage: $N^{1+o(1)}$.
- Total online time: $N^{o(1)}$.
- Total online communication: $N^{o(1)}$.

Proof. We prove the properties separately.

Correctness: For correctness, we simply need to argue that $f(0) = P(i)$. This is simple since we have $dm + 1$ distinct points and are recovering a function f of degree dm . Follows as in [78].

Public Parameters (set according to the proof of Theorem 2):

1. m : number of variables.
2. d : total degree of a polynomial function.
3. q : a prime power p^α for some prime p and $\alpha \in \mathbb{Z}^+$.
4. $k = dm + 1$: number of servers.
5. $\text{encode}(i) \rightarrow \mathbb{F}_q^m$: deterministic mapping for $i \in [N]$ to $\mathbb{Z}_q^m$.

Preprocess(DB):

1. Interpolate m -variate, d -degree polynomial $P : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$, such that $P(\text{encode}(i)) = \text{DB}[i]$.
2. Compute $\tilde{P} \leftarrow \text{PolyPreProcess}(P)$.
3. Return $\widetilde{\text{DB}} = \tilde{P}$.

Query($i \in [N], j \in [k] \subset \mathbb{F}_q$):

1. Sample $\vec{v} \leftarrow \mathbb{F}_q^m$.
2. Compute $q_j \leftarrow j \cdot \vec{v} + \text{encode}(i)$. (For simplicity, we view $[k]$ as the first k elements in $\mathbb{F}_q$.)
3. Return q_j .

Answer($\widetilde{\text{DB}}, q$) :

1. Return $\tilde{P}(q)$.

Reconstruct($a_1, \dots, a_k$) :

1. Use the answer to interpolate a function $f : \mathbb{F}_q \rightarrow \mathbb{F}_q$ of degree $d \cdot m$, using $f(j) = P(j \cdot \vec{v} + \text{encode}(i))$ (included in A) for $j \in [k]$ where $k = dm + 1$.
2. Return $f(0)$.

Fig. 1. Our multi-server doubly efficient PIR scheme.

Privacy: Privacy is also simple: since $\vec{v}$ is sampled uniformly at random from $\mathbb{F}_q^m$, $j \cdot \vec{v} + \text{encode}(i)$ is indistinguishable from a uniformly randomly drawn vector from $\mathbb{F}_q^m$ for any $j \in [k]$ (for $[k]$ denoting k elements of $\mathbb{F}_q$).

Efficiency: Now, to prove the efficiency, we first set the parameters accordingly.

1. We set $d^m = N$ and $d = \log \log(N) = 2^{\log \log \log(N)}$, which gives $m = O(\log(N)/\log \log \log(N))$.
2. Then, we set $p = O(\log \log \log(N)) = d^{o(1)}$ (satisfying the requirement in Theorem 1).
3. Lastly, we set α such that $p^\alpha = q \geq k + 1$, and thus:

$$\alpha = O(\log_p(k)) = O(\log \log(N)^{1/\log \log \log \log(N)}).$$

With these parameters, we analyze each efficiency metrics separately.

- Number of servers:

$$\begin{aligned} k &= m \cdot d \\ &= O(\log(N) \log \log(N) / \log \log \log(N)) \\ &= \tilde{O}(\log(N)) \end{aligned}$$

- Preprocessing time: Let's start with the preprocessing of the polynomial (line 2). Recall that from Theorem 1, the preprocessing time for a degree- d and m -variate polynomial is: $(\alpha dp)^m \cdot 4^m \cdot \text{poly}(\alpha, d, m, p)$. We analyze this term by term.

- $\alpha^m = (\log_p(k + 1))^m = O(N^{1/\log \log \log \log(N)}) = N^{o(1)}$.
- $d^m = N$.
- $p^m = N^{\log \log \log \log(N) / \log \log \log \log(N)} = N^{o(1)}$.
- $\text{poly}(\alpha, d, m, p) = \text{polylog}(N)$.

Therefore, the total preprocessing time for a single function is $N^{1+o(1)}$. For $dm + 1 = \text{polylog}(N)$ servers, The total polynomial preprocessing time is thus $N^{1+o(1)}$.

Now, let us move to the interpolation of the functions (line 1). As introduced in Lemma 1, the interpolation time is $N \text{polylog}(N)$. Putting all these together, the preprocessing time is $N^{1+o(1)}$. Therefore, the extra storage (i.e., $|\text{DB}|$) is also trivially bounded by $N^{1+o(1)}$.

- Online time: The online time of the client query is simply the time for encoding, which is $O(\log(N))$. Then, for each server, the online time is to evaluate the preprocessed function. As in Theorem 1, each evaluation takes $4^m \cdot \text{poly}(\alpha, d, m, p)$. Since $4^m = N^{o(1)}$ and $\text{poly}(\alpha, d, m, p) = \text{polylog}(N)$, the runtime of each server is $N^{o(1)}$. Since there are only $dm + 1$ servers, the total runtime is also $N^{o(1)}$. Lastly, the reconstruction time is the time to reconstruct f plus the evaluation of f , which is simply bounded by $\text{poly}(dm) = \text{polylog}(N)$ [29]. The evaluation is also $O(dm) = \text{polylog}(N)$ since the function has degree dm .

Note that the unit of the computation cost above is the number of $\mathbb{F}_q$ operations. Note that since $q = O(\text{polylog}(N))$, each $\mathbb{F}_q$ operation costs at most $\text{polylog}(N)$ bit operations and thus does not affect the conclusion above.

- Online communication: The query size is simply m $\mathbb{F}_q$ elements, and thus $\text{polylog}(N)$. The answer size is bounded by the total runtime of all the servers, which is thus $N^{o(1)}$. Thus, the total online communication is also $N^{o(1)}$.

With all these efficiency metrics proven, our construction indeed satisfies the definition of being *doubly efficient*.

5 Our Second Construction

Next, we explore whether we reduce the number of servers needed even further.

Recall that in [78], each server evaluates not only the interpolated function, but also the derivatives of the functions. While they only use one level of derivatives, we can naturally extend their idea to work over L levels of derivatives. In more detail, [78] shows that for an (odd) degree $2k - 1$ univariate function f , it suffices to have k unique points $f(x_i) = y_i$ for $i \in [k]$ and k unique derivatives $f'(x_i) = z_i$ to uniquely determine f . We further extend their insight and show that with k points, k first derivatives, k second derivatives, and so on, up to k L -th derivatives, to uniquely determine the function with degree $k(L + 1) - 1$, as formally proven in the following lemma.

Lemma 3. *Suppose $\{x_i\}, \{y_{0,i}\}, \dots, \{y_{L,i}\}$ are elements of some finite field $\mathbb{F}_q$, where $i \in [I]$ for some I greater than 0 and the set $\{x_i\}$ has size I ; then there exists at most one polynomial $f(x) \in \mathbb{F}_q[x]$ of degree less than or equal to $(L + 1) \cdot I - 1$ such that $f(x_i) = y_{0,i}$ and $f^\ell(x_i) = y_{\ell,i}$ for $\ell \in [L - 1]$ and f^ℓ denotes the ℓ -th derivative of f , where $\mathbb{F}_q$ has characteristics p greater than or equal to $L + 1$.*

Proof. If there exist two such polynomials $f_1(x), f_2(x)$, then let $f = f_1 - f_2$. Then, it is clear that $f(x_i) = \dots = f^L(x_i) = 0$ for all $i \in [I]$. Thus, by Lemma 2, it holds that $\prod_{i \in [I]} (x - x_i)^{L+1} | f(x)$, implying $f(x) = 0$ since the degree of f is at most $|L + 1|I - 1$.

With this in mind, instead, each server calculates the partial derivatives of the interpolated polynomial P , and preprocess these derivatives as a polynomial as well. More formally, each server computes

$$\frac{\partial^\ell P}{\partial x_{z_1} \dots \partial x_{z_\ell}},$$

for all $z_1, \dots, z_\ell \in [m]^\ell$.

With L levels of derivatives, we then only need approximately dm/L servers. However, note that for each level of derivative, the number of functions that are stored at the server grows by a factor of m (as $z_1, \dots, z_\ell \in [m]^\ell$). Thus, for L

levels, there are $O(m^L)$ functions. To make the preprocessing time and online costs reasonable, we need to tune the parameters accordingly.

In Theorem 3, we show that with the careful parameter setup, we can achieve $o(\log(N))$ servers, while maintaining the online cost to be $N^{o(1)}$ and preprocessing time to be $\text{poly}(N)$.

The construction of our recursive scheme is presented below in Figure 2.

Theorem 3 (Recursive dePIR). *The Private Information Retrieval scheme in Figure 2 satisfies Definition 1 that is query efficient. In particular, it runs with efficiencies:*

- Number of servers: $o(\log(N))$.
- Preprocess time and server storage: $N^{2+o(1)}$.
- Total online time: $N^{o(1)}$.
- Total online communication: $N^{o(1)}$.

Proof. We prove each of the properties separately. We start with the privacy and efficiency, since the correctness depends on the parameters setup analyzed in detail in the efficiency analysis.

Privacy: Privacy follows by exactly the same argument as in Theorem 2.

Efficiency: Now, to prove the efficiency, we first set the parameters accordingly.

1. We set $d^d = N$ which gives $d = \frac{\log(N)}{W(\log(N))}$ where $W(\cdot)$ is the Lambert W function [18].
2. Then, we set $m = d$.
3. And we set $L = \frac{\log(N)}{W(\log(N))^{1.5}}$.
4. Then, set $k = m \cdot d/L$.
5. Furthermore, we set prime p to be the smallest prime such that $p \geq L + 1$ (by Bertrand's postulate [67], $p = O(L)$).
6. Lastly, we set α such that $p^\alpha = q \geq k + 1$, which means:

$$\begin{aligned}
\alpha &= O(\log_p(k)) \\
&= \log_p(p \cdot W(\log(N))) \\
&= 1 + \log_p(W(\log(N))) \\
&= 1 + o(1) = O(1).
\end{aligned}$$

With these parameters, we analyze the efficiency metrics accordingly.

- Number of servers: $k = m \cdot d/L = O(\frac{\log(N)}{\sqrt{W(\log(N))}}) = o(\log(N))$.
- Preprocessing time: Let's start with the preprocessing of each polynomial (line 2 and 4). Recall that from Theorem 1, the preprocessing time for a single function is: $(\alpha dp)^m \cdot 4^m \cdot \text{poly}(\alpha, d, m, p)$. We analyze this term by term.
 - $\alpha^m = O(1)^m = N^{o(1)}$.

Public Parameters (set according to the proof of Theorem 3):

1. m : number of variables.
2. d : total degree of a polynomial function.
3. q : a prime power p^α for some prime p and $\alpha \in \mathbb{Z}^+$.
4. k : number of servers.
5. $\text{encode}(i) \rightarrow \mathbb{F}_q^m$: deterministic mapping for $i \in [N]$ to $\mathbb{Z}_q^m$.
6. L : levels of derivatives we need.

Preprocess(DB):

1. Interpolate m -variate, d -degree polynomial $P : \mathbb{F}_q^m \rightarrow \mathbb{F}_q$, such that $P(\text{encode}(i)) = \text{DB}[i]$.
 2. Compute $\tilde{P} \leftarrow \text{PolyPreProcess}(P)$.
 3. Compute $P_{x_{z_1}, \dots, x_{z_\ell}, \ell} \leftarrow \frac{\partial^\ell P}{\partial x_{z_1}, \dots, \partial x_{z_\ell}}$ for every $\ell \in [L]$, for all $(z_1, \dots, z_\ell) \in [m]^\ell$.
 4. Compute $\tilde{P}_{x_{z_1}, \dots, x_{z_\ell}, \ell} \leftarrow \text{PolyPreProcess}(P_{x_{z_1}, \dots, x_{z_\ell}, \ell})$.
 5. Return $\widetilde{\text{DB}} = (\tilde{P}, \tilde{P}_{x_{z_1}, \dots, x_{z_\ell}, \ell})$ for all $(z_1, \dots, z_\ell) \in [m]^\ell$ and $\ell \in [L]$.
-

Query($i \in [N], j \in [k] \subset \mathbb{F}_q$):

1. Sample $\vec{v} \leftarrow \mathbb{F}_q^m$.
 2. Compute $q_j \leftarrow j \cdot \vec{v} + \text{encode}(i)$. (For simplicity, we view $[k]$ as the first k elements in $\mathbb{F}_q$.)
 3. Return q_j .
-

Answer($\widetilde{\text{DB}}, q$) :

1. Return $\tilde{P}(q)$ and $\tilde{P}_{x_{z_1}, \dots, x_{z_\ell}, \ell}(q)$ for all $(z_1, \dots, z_\ell) \in [m]^\ell$ and $\ell \in [L]$.
-

Reconstruct($a_1, \dots, a_k$) :

1. Use the answer to interpolate a function $f : \mathbb{F}_q \rightarrow \mathbb{F}_q$ of degree $d \cdot m$, as follows:
 $f(j) = P(j \cdot \vec{v} + \text{encode}(i))$ (included in a_j); and

$$f^\ell(j) = \sum_{z_1, \dots, z_\ell \in [m]^\ell} \frac{\partial^\ell P}{\partial x_{z_1}, \dots, \partial x_{z_\ell}}(j \cdot \vec{v} + \text{encode}(i)) \prod_{i \in [\ell]} \vec{v}_{z_i} \quad (1)$$

2. Return $f(0)$.

Fig. 2. Our recursive PIR scheme.

- $d^m = N$ as parametrized above.
- $p^m = O(N)$ (since $m^m = O(N)$, $p = O(L)$ and $L = o(m)$).
- $\text{poly}(\alpha, d, m, p) = \text{polylog}(N)$.

Therefore, the total preprocessing time for a single function is $N^{2+o(1)}$. In total, there are at most:

$$m^L + 1 = O(N^{1/\sqrt{W(\log(N))}}) = N^{o(1)}$$

functions. Then, the total preprocessing time is $N^{2+o(1)}$.

Now, let us move to the interpolation of the functions and the calculations of the derivatives (line 1 and 3). As introduced in Lemma 1, the interpolation time is $N \cdot \text{polylog}(N)$. Furthermore, computing the partial derivative over a single variable of a function is trivially bounded by $O(N)$, and since there are at most $N^{o(1)}$ partial derivatives (as calculated above), the cost is $N^{1+o(1)}$. Putting all these together, the preprocessing time is $N^{1+o(1)}$. Therefore, the extra storage is also trivially bounded by $N^{2+o(1)}$.

- Online time: The online time of the client query is trivially the time for encoding, which is $\log(N)$. Then, for each server, the online time is to evaluate the preprocessed function. As in Theorem 1, each evaluation takes $4^m \cdot \text{poly}(\alpha, d, m, p)$. Since $4^m = N^{o(1)}$ and $\text{poly}(\alpha, d, m, p) = \text{polylog}(N)$, and there are a total of $m^L = N^{o(1)}$ evaluations, the runtime of each server is $N^{o(1)}$. Since there are only $o(\log(N))$ servers, the total runtime is also $N^{o(1)}$. Lastly, the reconstruction time is the time to reconstruct f plus the evaluation of f . Note that there are at most $N^{o(1)}$ points, we can thus recover $f(j), f'(j), f''(j), \dots, f^L(j)$ within time $N^{o(1)}$. Lastly, using n points, we can recover function f with time $\text{poly}(n)$ [29], and $n = N^{o(1)}$. The evaluation is also $O(dm) = \text{polylog}(N)$ since the function has degree dm .

Again, the unit of the computation cost above is the number of $\mathbb{F}_q$ operations. However, since $q = O(\text{polylog}(N))$, each $\mathbb{F}_q$ operation costs at most $\text{polylog}(N)$ bit operations and thus does not affect the calculation above.

- Online communication: The query size is simply m $\mathbb{F}_q$ elements, and thus $\text{polylog}(N)$. The answer size is bounded by the total runtime of all the servers, which is $N^{o(1)}$. Therefore, the total online communication for the scheme is also $N^{o(1)}$.

Correctness: Lastly, with the parameters setup, we need to argue that $f(0) = P(i)$. It is a simple application of chain rule as described in Section 3.

As shown in Lemma 3 (note that we set $p \geq L + 1$ and thus $\mathbb{F}_q$ has characteristics $p \geq L + 1$), since we have k unique points of f , k corresponding first derivatives of f , and so on, up to k corresponding L -th derivatives of f , we can uniquely determine a function f with degree $d \cdot m \leq k(L + 1) - 1$. The derivatives are obtained by a simple application of the chain rule (Eq. (1)).

Therefore, $f(0) = P(\text{encode}(i))$.

6 Multi-server PIR from Distributed Oblivious RAM

Now, we discuss a different model for multi-server PIR. As mentioned, to the best of our knowledge, all the existing works on multi-server PIR assume that the servers do not communicate with each other and do not need to update their states per query. While this gives a weaker environment assumption, we believe that it might be too restrictive, especially since that, for the reasons we argued in Section 1, it is hard to fully avoid server communication and keeping states.

Given the relaxation on communication and statefulness, we show that we can achieve PIR with much better efficiency: specifically, with only 3 servers, we can achieve information-theoretic PIR with $O(\log^3(N))$ online runtime and communication with quasi-linear preprocessing.⁴

Note that for statefulness, while the servers keep a state, they do not keep a state per client, but instead keep a global state for all clients. We believe that this is a more reasonable relaxation in practice since keeping a database, especially a frequently updated database, is a global state as well. This also draws a clear difference from Oblivious RAM (ORAM), which keeps a state *per client* (and moreover requires a database per client, causing $\Omega(N \cdot M)$ storage for M clients).

The asymptotics we can achieve above, given such relaxation, dominate what can be achieved by all existing multi-server PIR constructions (including even the new ones we proposed above). The large discrepancy comes first from the number of servers, which even given a “weak” dependence on N such as $O(\log N)$ or $o(\log N)$ can be hard to satisfy for any real system. Furthermore, looking ahead, the building blocks we rely on are relatively lightweight, so the hidden constants are relatively small, and therefore this path empirically leads to very practical PIR constructions. We discuss this briefly later in the section.

Now, we will go through how to actually instantiate the PIR construction. We start from achieving our desired PIR using a trusted third party, as it helps with exposition, then show how to get rid of it.

Starting point: a trusted third party. Our key observation starts from the fact that if there exists a trusted third party, PIR can be realized using Oblivious RAM (ORAM) [34].⁵ Specifically, given a server $\mathcal{S}$ and trusted party $\mathcal{T}$, we can realize PIR (with global preprocessing) as follows: the trusted party download the database DB with N entries (e.g., N bits) held by the server, encode the database into an ORAM database DB' and store it on $\mathcal{S}$ using ORAM. For a client $\mathcal{C}$, with query qry_i , send this query qry_i to $\mathcal{T}$. Then, $\mathcal{T}$ decrypts, performs an ORAM read query with $\mathcal{S}$ to obtain $\text{DB}[\text{qry}_i]$, and then sends $\text{DB}[\text{qry}_i]$ to $\mathcal{C}$. Clearly, this achieves the correctness requirement of PIR. Furthermore, privacy is guaranteed: ORAM guarantees that $\mathcal{S}$ *does not* learn anything about qry_i .⁶

⁴ This can be further reduced to only $\omega(\log^2(N))$ [25] but we use $O(\log^3(N))$ for simplicity.

⁵ For completeness, we include a more formal description of ORAM in Appendix B.

⁶ Note that a normal ORAM only protects access patterns assuming the server does not see the database contents themselves, which is easily achievable by having $\mathcal{T}$ encrypting the database and performing ORAM queries using re-randomized ci-

Removing the trusted third party. Then, the next step is to remove the trusted third party. Fortunately, using secure multi-party computation (MPC) [79,33], we can replace the trusted third party with a set of parties that can securely compute the function (given certain assumptions on their behavior, to be discussed later).

Specifically, if we replace the trusted party $\mathcal{T}$ with a set of k parties running MPC, acting as the ORAM client, we immediately obtain a $(k + 1)$ -server PIR. Of course, the issue is that the circuit of the MPC for ORAM query fetch is unclear: it may have size linear in N , which means that the online runtime is linear in N , making this scheme moot. However, in fact, we can indeed find a specialized MPC primitive to realize this efficiently, called *distributed ORAM*.

At a high-level, distributed ORAM (DORAM) [62] has k servers, a RAM functionality on secret-shared inputs and outputs. In more detail, DORAM participants (the k servers) jointly hold a database DB (each server holding a share of DB), and also some secret-shared index i , DORAM returns the shares of $\text{DB}[i]$ to each server. More formally, it is defined in Fig. 3, adapted from [24]:

Public parameters: database size $\text{DB} \in \{0,1\}^N$. k servers.

Procedures:

- $[\text{DB}'] \leftarrow \text{Init}([\text{DB}])$: initialize a secret-shared encoded database DB' with the initial database DB secret shared among the k servers.
- $[\text{DB}[i]], [\text{DB}] \leftarrow \text{Read}([i], [\text{DB}'])$: takes the encoded database DB' (secret-shared among k servers) and a secret-shared index i , and returns (1) the secret shares of $\text{DB}[i]$ (the i -th item of the original database) and (2) the updated secret-shared encoded database DB' to each server.

Fig. 3. Distributed ORAM Functionality (read-only). $[x]$ denotes the secret shares of x among the k servers. For simplicity, assume this is additive secret sharing. See Appendix B for details on secret sharing.

Note that we only need the read functionality of DORAM, so in Fig. 3, we only define the **Read** functionality. Despite the write functionality not being useful for our standard PIR functionality, it can be useful for updating database values or adding new database values in the settings where the database is dynamic. We discuss this further down in the section.⁷

phertexts. This is widely known, so we directly assume the ORAM does not leak access patterns even when $\mathcal{S}$ can see the (encoded) database.

⁷ The definition in [24] includes an **Extract** procedure (which extracts the original non-encoded database in the secret-sharing form) to be run periodically to re-instantiate the database for efficiency. This is not necessary in all DORAM constructions and is not relevant in the PIR setting.

From DORAM to PIR. Now, we are ready to discuss how to achieve PIR with DORAM. The construction is straightforward: we let the k servers secret share their (shared) database, and use DORAM Init to encode the database and obtain a shared $[\text{DB}']$. Then, for a client with query qry , it secret shares the query, sends the shares to each server. Then, the servers use DORAM to obtain the shares of $\text{DB}[\text{qry}]$ (and of course the updated encoded database DB' accordingly), and sends these shares back to the client. The client locally recovers $\text{DB}[\text{qry}]$ using the secret shares. We describe it formally as follows.

Public Parameters: 1. N: database size 2. k: number of servers. Preprocess(DB): 1. Server $j \in [k]$ obtains the share $[\text{DB}]_j$ (denoting the j-th share of DB). 2. All k servers together run $[\text{DB}'] \leftarrow \text{Init}([\text{DB}])$ (the DORAM initialization procedure) and server j stores $\widetilde{\text{DB}}_j := [\text{DB}']_j$. Query($i \in [N]$): 1. Secret share i into $[i]_j$ for server $j \in [k]$. 2. $q_j := [i]_j$. Answer($\widetilde{\text{DB}}_1 = [\text{DB}']_1, \dots, \widetilde{\text{DB}}_k = [\text{DB}']_k, q_1 = [i]_1, \dots, q_k = [i]_k$): 1. k servers together run $[\text{DB}[i]] \leftarrow \text{Read}([\text{DB}'], [i])$ and server j holds $[\text{DB}[i]]_j$. 2. $a_j := [\text{DB}[i]]_j$. Reconstruct($a_1 = [\text{DB}[i]]_1, \dots, [\text{DB}[i]]_k$): 1. Reconstruct $\text{DB}[i]$ via $[\text{DB}[i]]$. 2. Return $\text{DB}[i]$.
--

Fig. 4. Our DORAM-based PIR construction.

Theorem 4 (DORAM-based PIR). *The Private Information Retrieval scheme in Figure 4 satisfies Definition 2. In particular, it runs with efficiencies:*

- Number of servers: k .
- Preprocess time and server storage: T_{Init} .
- Total online time and communication: T_{Read} .

assuming the underlying DORAM scheme (Fig. 3) with k servers, **Init** runtime being T_{Init} , and **Read** runtime being T_{Read} .

Proof. The proof is straightforward. The correctness and privacy follow directly from DORAM and secret sharing. The efficiency also directly follows since the construction essentially only uses DORAM.

Realization and efficiency of DORAM. Since our PIR construction essentially directly follows from DORAM, an efficient DORAM construction implies an efficient PIR construction. Specifically, using GigaDORAM [25], we achieve quasi-linear preprocessing time, and $O(\log^3(N))$ online time and communication. These are asymptotically better than any existing multi-server PIR construction, including our constructions above and the concurrent work [30]. Concretely, this in fact allows us to make one PIR query against 8GB of database with only about 1ms, according to the benchmarks in [25], which is also orders of magnitude faster than any existing PIR construction. While GigaDORAM is based on computational privacy, we can also use other DORAM constructions with information-theoretic privacy achieving similar asymptotics (e.g., [24]). Malicious security can also be achieved by using [38]. Any future advances in DORAM could also greatly improve our PIR construction.

Updatable PIR. As mentioned, many PIR works are also concerned with the updates [48,63,57,28,54,43]. Luckily, our construction can deal with updates very efficiently. Specifically, DORAM, in addition, supports writes as well, which means that the servers can simply jointly use DORAM writes to update the database with $O(\log^3(N))$ time.

Further application to Oblivious Message Retrieval. An additional application of DORAM is Oblivious Message Retrieval (OMR) [59]. Essentially, OMR works in anonymous messaging systems, where the recipient would like to retrieve the message address to them, but for a message, before performing the retrieval, only the sender knows who the recipient is, and this information should be kept confidential from anyone else. To address this issue, most existing construction as single-server [59,60,61,58,56]. There are also several two-server constructions [46,17]. However, all these constructions have *linear* online time per retrieval per recipient, even for the two-server constructions.

However, this online time can be greatly improved with DORAM. Specifically, in [44], the authors introduced a way to build OMR with trusted hardware plus ORAM. One could easily replace this trusted hardware with MPC and ORAM with DORAM. This allows a multi-server OMR construction with poly-logarithmic online time, greatly improving upon the state-of-the-art constructions asymptotically. However, whether this approach is concretely more efficient is still an open question left for future work.

7 Concluding Remarks

In this paper, we introduce three multi-server PIR constructions. Our first two constructions are more of theoretical interests, while our third construction, we believe, for the very first time, introduces a new path towards making PIR practical. As mentioned, with existing DORAM constructions, we can fetch one PIR query with only $\approx 1\text{ms}$ for an 8GB database, and the runtime growth is only $O(\log^3(N))$.

Rational instead of honest servers. One interesting question to consider is the case where the servers might want to collude covertly, if it is profitable to them. In particular, as all prior works [78,8,30], our work assumes that the servers do not collude.

However, one interesting direction is to relax this assumption further. Instead of assuming all the servers to be semi-honest and not to collude with each other, can we relax this assumption to assume the servers to be rational players? In this framework, we need to prove that any rational server would not want to collude and instead follow the protocol.

A recent work [35] explored this multi-server PIR in this rational setting. At a high level, the work provides a transformation from the non-collusion model to the rational server model through a public contract. It works as follows.

When servers collude and learn any non-trivial information $f(i)$ where i is the index that the client retrieves, within a pre-determined period of time (which can be long, e.g., years), the server(s) can submit a report to show that they have learned such nontrivial information via collusion. The first server that correctly submits the report is rewarded and the rest gets punished. To avoid false reports and false accusations, the work employs additional techniques.

This same idea can be applied to our multi-server PIR constructions in practice to avoid the non-collusion assumption. According to [35], the cost of setting it up is under 10 dollars when setting up the contract using Ethereum.

Future work. For future work, we find the following questions to be compelling:

- (Reducing the number of servers) For the non-communicating model, can we reduce the number of non-colluding servers required to a constant number (ideally 2), while maintaining our current asymptotics (or slightly relaxing the current asymptotics, e.g., allowing the preprocessing time to be $O(N^{1+\epsilon})$ for some constant ϵ)? We know of no negative results in this regard. Nevertheless, there are no known constructions even given some that achieve even close to the asymptotics above with a small, constant number of servers.
- (Computational PIR) For the non-communicating setting, can introducing (lightweight) computational assumptions (e.g. OWF) help reduce either the number of servers or the online query time for our family of schemes, without ruining the practicality? In non-preprocessing PIR, distributed point functions have enabled two-server computational PIR with $O(\log N)$ communication, which has yet to be achieved for two-server information-theoretic PIR. We think it is probable that likewise, some computational assumption

may help further improve the current known asymptotics for multi-server server-preprocessing PIR.

- (Relaxing the non-collusion assumption) All multi-server PIR schemes require the assumption that the participating servers are non-colluding. However, when allowing for communication, it is not clear why we cannot achieve constructions assuming only 1-out-of- k honest party (or more generally t -out-of- k for some $t < k$). Thus, one interesting open question is to construct DORAM with $k > 3$ assuming only 1 or $t \ll k$ honest parties without affecting the scheme’s efficiency, which directly affects our PIR construction.

Acknowledgements

We thank Ran Canetti and Ling Ren for helpful discussions on alternative models for multi-server PIR. Charalampos Papamanthou and Arthur Lazzaretti were supported by NSF and Protocol Labs. Zeyu Liu was supported by Yale CADMY. Peihan Miao was supported in part by NSF, Meta, Google, and Amazon AWS.

References

1. I. Ahmad, Y. Yang, D. Agrawal, A. E. Abbadi, and T. Gupta. Addra: Metadata-private voice communication over fully untrusted infrastructure. 2021.
2. A. Ali, T. Lepoint, S. Patel, M. Raykova, P. Schoppmann, K. Seth, and K. Yeo. {Communication–Computation} Trade-offs in {PIR}. pages 1811–1828, 2021.
3. S. Angel and S. Setty. Unobservable communication over fully untrusted infrastructure. In *Proceedings of the 12th USENIX conference on Operating Systems Design and Implementation*, OSDI’16, pages 551–569, USA, Nov. 2016. USENIX Association.
4. M. Backes, A. Kate, M. Maffei, and K. Pecina. ObliviAd: Provably Secure and Practical Online Behavioral Advertising. In *2012 IEEE Symposium on Security and Privacy*, pages 257–271, May 2012. ISSN: 2375-1207.
5. A. Beimel and Y. Ishai. Information-Theoretic Private Information Retrieval: A Unified Construction. In G. Goos, J. Hartmanis, J. Van Leeuwen, F. Orejas, P. G. Spirakis, and J. Van Leeuwen, editors, *Automata, Languages and Programming*, volume 2076, pages 912–926. Springer Berlin Heidelberg, Berlin, Heidelberg, 2001.
6. A. Beimel, Y. Ishai, E. Kushilevitz, and J.-F. Raymond. Breaking the $O(n(2k-1)/\epsilon)$ barrier for information-theoretic Private Information Retrieval. pages 261–270, Feb. 2002.
7. A. Beimel, Y. Ishai, and T. Malkin. Reducing the Servers Computation in Private Information Retrieval: PIR with Preprocessing. In *CRYPTO 2000*. Springer, 2000.
8. A. Beimel, Y. Ishai, and T. Malkin. Reducing the servers’ computation in private information retrieval: Pir with preprocessing. *Journal of Cryptology*, 17:125–151, 2004.
9. A. Beimel and Y. Stahl. Robust Information-Theoretic Private Information Retrieval. *Journal of Cryptology*, 20(3):295–321, July 2007.
10. V. Bhargava, S. Ghosh, Z. Guo, M. Kumar, and C. Umans. Fast Multivariate Multipoint Evaluation Over All Finite Fields. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 221–232, Oct. 2022. ISSN: 2575-8454.
11. V. Bhargava, S. Ghosh, M. Kumar, and C. K. Mohapatra. Fast, algebraic multivariate multipoint evaluation in small characteristic and applications. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, pages 403–415, New York, NY, USA, June 2022. Association for Computing Machinery.
12. E. Boyle, N. Gilboa, and Y. Ishai. Function Secret Sharing. In E. Oswald and M. Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015*, Lecture Notes in Computer Science, pages 337–367, Berlin, Heidelberg, 2015. Springer.
13. E. Boyle, Y. Ishai, R. Pass, and M. Wootters. Can We Access a Database Both Locally and Privately? Nov. 2017.
14. B. Chor, N. Gilboa, and M. Naor. Private Information Retrieval by Keywords, 1998. Report Number: 003.
15. B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan. Private information retrieval. Oct. 1995.
16. B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan. Private information retrieval. In *36th FOCS*, pages 41–50. IEEE Computer Society Press, Oct. 1995.
17. H. Chu, X. Wang, and Y. Jia. Private signaling secure against actively corrupted servers. Cryptology ePrint Archive, Paper 2025/1056, 2025.

18. R. Corless, G. Gonnet, D. Hare, D. Jeffrey, and D. Knuth. On the lambert w function. *Advances in Computational Mathematics*, 5:329–359, 01 1996.
19. H. Corrigan-Gibbs, A. Henzinger, and D. Kogan. Single-Server Private Information Retrieval with Sublinear Amortized Time. In *EUROCRYPT 2022*, Berlin, Heidelberg, May 2022. Springer-Verlag.
20. H. Corrigan-Gibbs and D. Kogan. Private Information Retrieval with Sublinear Online Time. In A. Canteaut and Y. Ishai, editors, *EUROCRYPT 2020*, Cham, 2020. Springer International Publishing.
21. J. Doerner and a. shelat. Scaling ORAM for secure computation. In B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, editors, *ACM CCS 2017*, pages 523–535. ACM Press, Oct. / Nov. 2017.
22. S. Faber, S. Jarecki, S. Kentros, and B. Wei. Three-party ORAM for secure computation. In T. Iwata and J. H. Cheon, editors, *ASIACRYPT 2015, Part I*, volume 9452 of *LNCS*, pages 360–385. Springer, Berlin, Heidelberg, Nov. / Dec. 2015.
23. B. H. Falk, D. Noble, and R. Ostrovsky. 3-party distributed ORAM from oblivious set membership. In C. Galdi and S. Jarecki, editors, *SCN 22*, volume 13409 of *LNCS*, pages 437–461. Springer, Cham, Sept. 2022.
24. B. H. Falk, D. Noble, and R. Ostrovsky. MetaDORAM: Info-theoretic distributed ORAM with less communication. Cryptology ePrint Archive, Report 2024/011, 2024.
25. B. H. Falk, R. Ostrovsky, M. Shtepel, and J. Zhang. GigaDORAM: Breaking the billion address barrier. In J. A. Calandrino and C. Troncoso, editors, *USENIX Security 2023*, pages 3871–3888. USENIX Association, Aug. 2023.
26. B. Fisch, A. Lazzaretti, Z. Liu, and C. Papamanthou. Single server pir via homomorphic thorp shuffles. Cryptology ePrint Archive, Paper 2024/482, 2024. <https://eprint.iacr.org/2024/482>.
27. B. Fisch, A. Lazzaretti, Z. Liu, and C. Papamanthou. ThorPIR: Single server PIR via homomorphic thorp shuffles. In B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, editors, *ACM CCS 2024*, pages 1448–1462. ACM Press, Oct. 2024.
28. B. Fisch, A. Lazzaretti, Z. Liu, and C. Papamanthou. ThorPIR: Single Server PIR via Homomorphic Thorp Shuffles. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, pages 1448–1462, New York, NY, USA, Dec. 2024. Association for Computing Machinery.
29. J. V. Z. Gathen and J. Gerhard. *Modern Computer Algebra*. Cambridge University Press, USA, 2 edition, 2003.
30. A. Ghoshal, B. Li, Y. Ma, C. Dai, and E. Shi. Information-Theoretic Multi-Server PIR with Global Preprocessing. Cryptology ePrint Archive, Paper 2024/765, 2024. Version: 20240519:202013.
31. A. Ghoshal, B. Li, Y. Ma, C. Dai, and E. Shi. Scalable multi-server private information retrieval. Cryptology ePrint Archive, Paper 2024/765, 2024.
32. N. Gilboa and Y. Ishai. Distributed Point Functions and Their Applications. In P. Q. Nguyen and E. Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*. Springer, 2014.
33. O. Goldreich, S. Micali, and A. Wigderson. How to play ANY mental game. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing, STOC ’87*, pages 218–229, New York, NY, USA, Jan. 1987. Association for Computing Machinery.
34. O. Goldreich and R. Ostrovsky. Software protection and simulation on oblivious RAMs. *Journal of the ACM*, 43(3):431–473, May 1996.
35. T. Gong, R. Henry, A. Psomas, and A. Kate. Relax the non-collusion assumption for multi-server PIR. *S&P 2024*, 2024.

36. T. Gupta, N. Crooks, W. Mulhern, S. Setty, L. Alvisi, and M. Walfish. Scalable and private media consumption with Popcorn. In *Proceedings of the 13th Usenix Conference on Networked Systems Design and Implementation*, NSDI'16, pages 91–107, USA, Mar. 2016. USENIX Association.
37. A. Hamlin and M. Varia. Two-server distributed ORAM with sublinear computation and constant rounds. In J. Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 499–527. Springer, Cham, May 2021.
38. B. Hemenway, D. Noble, R. Ostrovsky, M. Shtepel, and J. Zhang. DORAM revisited: Maliciously secure RAM-MPC with logarithmic overhead. In G. N. Rothblum and H. Wee, editors, *TCC 2023, Part I*, volume 14369 of *LNCS*, pages 441–470. Springer, Cham, Nov. / Dec. 2023.
39. A. Henzinger, E. Dauterman, H. Corrigan-Gibbs, and N. Zeldovich. Private Web Search with Tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSPP '23, pages 396–416, New York, NY, USA, Oct. 2023. Association for Computing Machinery.
40. A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan. One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval. page 27, 2022.
41. L. Hetz, T. Schneider, and C. Weinert. Scaling Mobile Private Contact Discovery to Billions of Users, 2023. Publication info: Published elsewhere. Minor revision. ESORICS 2023.
42. J. Holmgren, R. Canetti, and S. Richelson. Towards Doubly Efficient Private Information Retrieval. Technical Report 568, 2017.
43. A. Hoover, S. Patel, G. Persiano, and K. Yeo. Plinko: Single-server PIR with efficient updates via invertible PRFs. Eurocrypt'25, 2024.
44. S. Jakkamsetti, Z. Liu, and V. Madathil. Scalable private signaling. CSF 2025, 2025.
45. S. Jarecki and B. Wei. 3PC ORAM with low latency, low bandwidth, and fast batch retrieval. In B. Preneel and F. Vercauteren, editors, *ACNS 2018*, volume 10892 of *LNCS*, pages 360–378. Springer, Cham, July 2018.
46. Y. Jia, V. Madathil, and A. Kate. HomeRun: High-efficiency oblivious message retrieval, unrestricted. CCS 2024, 2024.
47. K. S. Kedlaya and C. Umans. Fast Polynomial Factorization and Modular Composition. *SIAM Journal on Computing*, 40(6):1767–1802, Jan. 2011. Publisher: Society for Industrial and Applied Mathematics.
48. D. Kogan and H. Corrigan-Gibbs. Private Blocklist Lookups with Checklist. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2021.
49. E. Kushilevitz and R. Ostrovsky. Replication is not needed: single database, computationally-private information retrieval. In *FOCS 1997*, 1997.
50. A. Lazaretti and C. Papamanthou. Near-optimal private information retrieval with preprocessing. In G. N. Rothblum and H. Wee, editors, *TCC 2023, Part II*, volume 14370 of *LNCS*, pages 406–435. Springer, Cham, Nov. / Dec. 2023.
51. A. Lazaretti and C. Papamanthou. Near-Optimal Private Information Retrieval with Preprocessing. In G. Rothblum and H. Wee, editors, *TCC 2023*, Cham, 2023. Springer Nature Switzerland.
52. A. Lazaretti and C. Papamanthou. TreePIR: Sublinear-time and polylog-bandwidth private information retrieval from DDH. In H. Handschuh and A. Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 284–314. Springer, Cham, Aug. 2023.

53. A. Lazzaretti and C. Papamanthou. TreePIR: Sublinear-Time and Polylog-Bandwidth Private Information Retrieval from DDH. In *CRYPTO 2023*. Springer-Verlag, 2023.
54. A. Lazzaretti and C. Papamanthou. Single Pass Client Preprocessing Private Information Retrieval. In *USENIX Security 2024*, 2024.
55. A. Lazzaretti and C. Papamanthou. Single pass client-preprocessing private information retrieval. In D. Balzarotti and W. Xu, editors, *USENIX Security 2024*. USENIX Association, Aug. 2024.
56. K. Lee and Y. Yeo. SophOMR: Improved oblivious message retrieval from SIMD-aware homomorphic compression. Cryptology ePrint Archive, Report 2024/1814, 2024.
57. W.-K. Lin, E. Mook, and D. Wichs. Doubly Efficient Private Information Retrieval and Fully Homomorphic RAM Computation from Ring LWE, 2022. Report Number: 1703.
58. Z. Liu, K. Sotiraki, E. Tromer, and Y. Wang. Snake-eye resistant PKE from LWE for oblivious message retrieval and robust encryption. In S. Fehr and P.-A. Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 126–156. Springer, Cham, May 2025.
59. Z. Liu and E. Tromer. Oblivious message retrieval. In Y. Dodis and T. Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 753–783, Cham, 2022. Springer Nature Switzerland. Full version: Cryptology ePrint Archive 2021; internal citations follow the latter’s numbering.
60. Z. Liu, E. Tromer, and Y. Wang. Group oblivious message retrieval. In *2024 IEEE Symposium on Security and Privacy*, pages 4367–4385. IEEE Computer Society Press, May 2024.
61. Z. Liu, E. Tromer, and Y. Wang. PerfOMR: Oblivious message retrieval with reduced communication and computation. In D. Balzarotti and W. Xu, editors, *USENIX Security 2024*. USENIX Association, Aug. 2024.
62. S. Lu and R. Ostrovsky. Distributed oblivious RAM for secure two-party computation. In A. Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 377–396. Springer, Berlin, Heidelberg, Mar. 2013.
63. Y. Ma, Z. Ke, T. Rabin, and S. Angel. Incremental Offline/Online PIR (extended version). In *USENIX Security 2022*, 2022.
64. S. J. Menon and D. J. Wu. Spiral: Fast, High-Rate Single-Server PIR via FHE Composition. In *IEEE Symposium on Security and Privacy, 2022*, 2022.
65. P. Mittal, F. Olumofin, C. Troncoso, N. Borisov, and I. Goldberg. {PIR-Tor}: Scalable Anonymous Communication Using Private Information Retrieval. 2011.
66. M. H. Mughees, H. Chen, and L. Ren. OnionPIR: Response Efficient Single-Server PIR. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security, CCS ’21*, pages 2292–2306, New York, NY, USA, Nov. 2021. Association for Computing Machinery.
67. T. Nagell. *Introduction to Number Theory*. American Mathematical Soc., 1964. Google-Books-ID: Znc5EAAAQBAJ.
68. H. Okada, R. Player, S. Pohmann, and C. Weinert. Towards Practical Doubly-Efficient Private Information Retrieval. *Financial Cryptography and Data Security 2024*, 2024.
69. H. Okada, R. Player, S. Pohmann, and C. Weinert. On Algebraic Homomorphic Encryption and Its Applications to Doubly-Efficient PIR. In *Advances in Cryptology – EUROCRYPT 2025*, pages 34–64. Springer, Cham, 2025. ISSN: 1611-3349.

70. S. Patel, G. Persiano, and K. Yeo. Private Stateful Information Retrieval. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, pages 1002–1019, New York, NY, USA, Oct. 2018. Association for Computing Machinery.
71. G. Persiano and K. Yeo. Limits of Preprocessing for Single-Server PIR. Technical Report 235, 2022.
72. L. Ren, M. H. Mughees, and I. Sun. Simple and practical amortized sublinear private information retrieval using dummy subsets. In B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, editors, *ACM CCS 2024*, pages 1420–1433. ACM Press, Oct. 2024.
73. A. Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, Nov. 1979.
74. E. Shi, W. Aqeel, B. Chandrasekaran, and B. Maggs. Puncturable Pseudorandom Sets and Private Information Retrieval with Near-Optimal Online Bandwidth and Time. In *Advances in Cryptology - CRYPTO*, 2021.
75. E. Stefanov, M. V. Dijk, E. Shi, T.-H. H. Chan, C. Fletcher, L. Ren, X. Yu, and S. Devadas. Path ORAM: An Extremely Simple Oblivious RAM Protocol. *J. ACM*, 65(4):18:1–18:26, Apr. 2018.
76. A. Vadapalli, R. Henry, and I. Goldberg. Duoram: A bandwidth-efficient distributed ORAM for 2- and 3-party computation. In J. A. Calandrino and C. Troncoso, editors, *USENIX Security 2023*, pages 3907–3924. USENIX Association, Aug. 2023.
77. Z. Wang and L. Ren. Single-server client preprocessing PIR with tight space-time trade-off. In S. Fehr and P.-A. Fouque, editors, *EUROCRYPT 2025, Part VI*, volume 15606 of *LNCS*, pages 94–122. Springer, Cham, May 2025.
78. D. Woodruff and S. Yekhanin. A Geometric Approach to Information-Theoretic Private Information Retrieval. 2005.
79. A. C. Yao. Protocols for secure computations. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science*, SFCS '82, pages 160–164, USA, Nov. 1982. IEEE Computer Society.
80. S. Yekhanin. Towards 3-query locally decodable codes of subexponential length. *Journal of the ACM*, 55(1):1–16, Feb. 2008.
81. S. Yekhanin. *Locally Decodable Codes and Private Information Retrieval Schemes*. Information Security and Cryptography. Springer, Berlin, Heidelberg, 2010.
82. S. Zahur, X. S. Wang, M. Raykova, A. Gascón, J. Doerner, D. Evans, and J. Katz. Revisiting square-root ORAM: Efficient random access in multi-party computation. In *2016 IEEE Symposium on Security and Privacy*, pages 218–234. IEEE Computer Society Press, May 2016.
83. L. Zhao, X. Wang, and X. Huang. Verifiable single-server private information retrieval from LWE with binary errors. *Information Sciences*, 546:897–923, Feb. 2021.
84. M. Zhou, W.-K. Lin, Y. Tselekounis, and E. Shi. Optimal Single-Server Private Information Retrieval. *ePrint IACR*, 2022.
85. M. Zhou, W.-K. Lin, Y. Tselekounis, and E. Shi. Optimal single-server private information retrieval. In C. Hazay and M. Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 395–425. Springer, Cham, Apr. 2023.
86. M. Zhou, A. Park, E. Shi, and W. Zheng. Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation, 2023. IEEE S&P 2024.

A Discussion on the Concurrent Work

A very recent work [30] also focuses on information-theoretic multi-server PIR with server-side preprocessing.

We first note that their work does not cover the case of communicating servers. It focuses solely on the setting of k non-colluding, non-communicating servers. From Table 1, we see that asymptotically, the results from Theorem 4 dominate the results from concurrent work, however, it requires servers to communicate at query time.

In terms of comparison between our work and their work for the setting of non-communicating servers, our works compare as follows. In [30, Thm 1.3 & Appendix B], they achieve $\text{poly}(N)$ preprocessing time and storage, $\text{polylog}(N)$ online costs (computation and communication) with $\text{polylog}(N)$ servers, which asymptotically is similar to [8, Thm 4.9] (summarized in Table 1, the fourth row).

Compared to our result in this non-communicating-server setting, our result achieves better preprocessing time than both schemes (with fewer servers), as seen in Table 1 (although our query time is slightly worse, it is not clear that either scheme can achieve our preprocessing time, even at the sacrifice of their query time).

The other results in [30] provide alternative constructions for a constant number of servers which allow finer tuning than prior works in terms of online computation and communication (see Table 1, the third row): allowing better communication by trading-off some online computation cost. On the other hand, our paper focuses on providing a tradeoff between the number of servers and the preprocessing time, for a reasonably small number of servers (roughly or less than $\log(N)$ servers), while still maintaining small online costs (i.e., $N^{o(1)}$).

In terms of techniques for this setting, our works share similarities as both are based on [78] and both use polynomial preprocessing. However, they use Hasse derivatives while we use normal derivatives, as well as different techniques and a different polynomial preprocessing scheme.

This comparison is with the version of their paper released concurrently with ours. A later version of their paper improves some of their results further. Specifically, newer version of [30] was after the original version and our original version in [31]. They achieve close to $N^{1+o(1)}$ preprocessing time and storage, close to $N^{o(1)}$ online time and communication, with $\omega(1)$ servers. This $o(1), \omega(1)$ asymptotics is a summary of all the results from [31], but for a more detailed representation, please see [31, Thm 1.1, 1.2, 1.3]. While this improves upon their older result, it is still dominated by our DORAM-based constructions. Thus, a similar conclusion can be drawn as before.

B Additional Preliminaries

Oblivious RAM. At a high level, Oblivious RAM (ORAM) [34] allows one to hide its access pattern to a database.

Definition 3. *ORAM contains one PPT algorithm:*

- $\text{data}' \leftarrow \text{Access}(\text{DB}, \text{op}, \text{a}, \text{data})$: on the input of an identifier DB to some database, an operation op that can be either **Read** or **Write**, an identifier of the data block (e.g., memory location of the data block), and a data block data , returns a data block data .

Let $\text{ORAM.Read}(\text{DB}, \text{a})$ denote $\text{Access}(\text{DB}, \text{Read}, \text{a}, \cdot)$ (here $\cdot$ is a dummy input) and let $\text{ORAM.Write}(\text{DB}, \text{a}, \text{data})$ denote $\text{Access}(\text{DB}, \text{Write}, \text{a}, \text{data})$. Let $\vec{x} = ((\text{DB}, \text{op}_1, \text{a}_1, \text{data}_1), \dots, (\text{DB}, \text{op}_Q, \text{a}_Q, \text{data}_Q))$ denote a data request sequence of length Q , where op_i is the operation that can be either **Read** OR **Write**, a_i is the identifier of the data block, and data_i is the data being written. We define ORAM construction's correctness and privacy as follows:

- (Correctness) On the input $\vec{x}$, it returns data that is consistent with $\vec{x}$ with probability $\geq 1 - \text{negl}(|\vec{x}|)$.
- (Computational Privacy) Let $A(\vec{x})$ be a sequence of accesses given the sequence of data requests $\vec{x}$. An ORAM construction is computationally private if for any two data request sequences $\vec{y}$ and $\vec{z}$ of the same length, their access patterns $A(\vec{y})$ and $A(\vec{z})$ are computationally indistinguishable by PPT algorithm except for the TEE.

As mentioned, ORAM does not consider the server's ability to see the database. However, as long as the database is encrypted and all the operations are done via rerandomized ciphertexts, the privacy guarantee is maintained even if the server can see the database. This is widely assumed as recognized in ORAM literature (e.g., [75]).

Secret sharing. As mentioned, we use secret sharing [73] in Section 6. For simplicity, aligned with the DORAM definition used in [24], we use additive secret sharing. Specifically, for $x \in \{0,1\}^*$, $x = \bigoplus_{j \in [k]} x_j$ where $x_j \leftarrow_{\$} \{0,1\}^*$ for $j \in [k-1]$. Then, $[x]_j := x_j$ is the j -th share of x .